

# Signhify — Local IDE Development Guide

Continue building Signhify in any local AI-native IDE (Google **Antigravity**, **Cursor**, **Windsurf**, **Kiro**, **Gemini CLI**, **Kilo CLI**, **Claude Code**, **Aider**, **Zed**, or plain **VS Code**) while keeping deployments shipping through **Lovable**. Last updated: June 8, 2026 · Maintained by Piyush Raj Singh · [hello@signhify.online](mailto:hello@signhify.online)

This project is bidirectionally synced with GitHub. Anything you push to the default branch is picked up by Lovable within seconds and becomes part of the next preview / publish. Anything you change in Lovable is committed straight back to GitHub. That is the whole contract — everything below is the practical detail to make it boringly reliable.

## 1. One-time setup

### 1.1 Prerequisites

| Tool                     | Min version | Why                                                                             |
|--------------------------|-------------|---------------------------------------------------------------------------------|
| Node.js                  | 20.x LTS    | Required by Vite 7 + TanStack Start                                             |
| Bun                      | 1.1+        | Package manager + script runner<br>( <code>bun.lockb</code> is source of truth) |
| Git                      | 2.40+       | GitHub sync                                                                     |
| GitHub account           | —           | Bidirectional sync with Lovable                                                 |
| Lovable workspace access | —           | To publish                                                                      |

**Do not** swap Bun for npm/pnpm/yarn. The `bun.lockb` file is the source of truth and Lovable's GitHub sync expects it. Mixing package managers produces lockfile drift and breaks the auto-deploy.

### 1.2 Connect Lovable ↔ GitHub (do this once)

In Lovable: **Plus (+) menu** → **GitHub** → **Connect project** → **Create Repository**.

Pick the org/account you want. Lovable creates the repo and pushes the current codebase.

### 1.3 Clone locally

```
git clone git@github.com:<your-org>/signhify.git
cd signhify
bun install
cp .env.example .env    # if present, otherwise create .env (see §2)
bun run dev
```

The dev server runs on `http://localhost:3000`. Hot reload, SSR, and TanStack

Router file-based routing all work the same as inside Lovable.

---

## 2. Environment variables

Two scopes. Never mix them.

### Browser / build-time (VITE\_\*)

Safe to bundle. Put in local `.env`:

```
VITE_SUPABASE_URL=https://<project>.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=eyJ...
```

### Server / runtime (process.env.\*)

Secrets. Read inside `handler()` of server functions only.

```
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_PUBLISHABLE_KEY=eyJ...
SUPABASE_SERVICE_ROLE_KEY=eyJ...    # never commit, never expose to client
LOVABLE_API_KEY=...                  # optional, for AI gateway
```

On Lovable Cloud the runtime secrets are injected automatically — your local `.env` is only for `bun run dev` on your machine. To rotate or add a runtime secret used in production, do it from the **Lovable** → **Cloud** → **Secrets** panel (not from GitHub).

---

## 3. IDE-specific setup

Pick whichever you use. All of them work because they only need (a) the filesystem and (b) `git`.

### 3.1 Cursor / Windsurf / Zed / VS Code

Open the cloned folder. Recommended extensions:

- Biome / Prettier (formatting already configured via `.prettierrc`)
- ESLint
- Tailwind CSS IntelliSense
- TanStack Router (optional — gives route-tree hints)

Cursor/Windsurf agents: point them at `CONTRIBUTING.md` and `DEPLOY.md` as "always-attached" context so they respect the project's conventions.

### 3.2 Google Antigravity

1. **File** → **Open Workspace** on the cloned repo.
2. In the Agent panel, add `CONTRIBUTING.md`, `DEPLOY.md`, and this guide to *Knowledge*.
3. Configure the **Run Task** integration to use `bun run dev` and `bun run prepublish:check`.
4. Antigravity's Browser Use mode targets `http://localhost:3000` — keep the dev server running while you iterate.

### 3.3 Kiro

1. `kiro init` inside the repo to register the spec workspace.
2. Create a spec at `.kiro/specs/feature-name/` for each new feature; Kiro will scaffold and edit routes under `src/routes/` following the TanStack Start conventions documented in `src/routes/README.md`.
3. Run `kiro hooks add prepublish` so Kiro runs `bun run prepublish:check` before any push to `main`.

### 3.4 Gemini CLI

```
npm install -g @google/gemini-cli
gemini auth login
cd signify
gemini                    # interactive session
# or one-shot
gemini -p "add a pricing FAQ section to /pricing"
```

Add a `GEMINI.md` at the repo root if you want to override the default context window. The CLI reads `CONTRIBUTING.md` automatically.

### 3.5 Kilo CLI

```
brew install kilo-cli      # or curl install per docs
kilo init
kilo run "implement <task>"
```

Kilo respects `.kilo/config.yaml`. Add `prepublish:check` as the pre-commit guard so failed Playwright runs block the commit.

### 3.6 Claude Code / Aider / OpenCode

Drop in at the repo root and go. They all use `git` for change tracking and will respect `bun.lockb`. For Aider:

```
aider --model claude-3-5-sonnet --read CONTRIBUTING.md DEPLOY.md
```

---

## 4. Day-to-day workflow

[illegible]

## 4.1 Branching

- Work on short-lived feature branches: `feat/<scope>` or `fix/<scope>`.
- Open a PR into `main`. Lovable previews track `main` by default; enable

*GitHub Branch Switching* in **Account** → **Labs** if you need per-branch previews.

- Squash-merge to keep the Lovable timeline readable.

## 4.2 Things you must never hand-edit

These are regenerated and will cause merge conflicts or runtime breakage:

- `src/routeTree.gen.ts`
- `src/integrations/supabase/types.ts`
- `src/integrations/supabase/client.ts`, `client.server.ts`,  
`auth-middleware.ts`, `auth-attacher.ts`
- `bun.lockb` (let bun install manage it)

### 4.3 Database changes

Always go through a migration file:

```
# create the file
touch supabase/migrations/$(date +%Y%m%d%H%M%S)_<name>.sql
# write SQL – remember GRANTS for any new public-schema table
```

Lovable picks up new migrations automatically on deploy. Never run ad-hoc SQL against production from your local shell.

#### 4.4 Server functions vs. server routes

- `createServerFn` (TanStack Start) for internal RPC — files in `src/lib/*.functions.ts`.
- `createFileRoute` with a `server` block for webhooks/public APIs — files in `src/routes/api/public/*`.

Do **not** add Supabase Edge Functions for new work — they're being phased out in favour of TanStack server functions.

## 5. Pre-publish checklist (mandatory)

Codified in `CONTRIBUTING.md` §8. The short version:

```
bun run prepublish:check
```

This runs the Playwright smoke + HTML-diff suite against the latest preview. It must pass before you publish from Lovable. If you publish without it, the `/publish` audit row will not be created and the gate will block subsequent ships.

Manual gates in addition to the script:

- `[]` Preview health: `/api/public/health` returns `{ok:true}`
- `[]` Marketplace diff is intentional (compare against `src/lib/marketplace.ts`)
- `[]` SEO: every changed route has unique title, description, OG + Twitter tags
- `[]` Security scan clean (no unresolved critical findings)
- `[]` Sync hygiene: `git status` clean, `bun.lockb` unchanged unless deps changed

---

## 6. Publishing

You have two equivalent paths:

1. **From Lovable** — open the project, click **Publish** (top-right on desktop, bottom-right on mobile in preview). Frontend changes deploy on click; backend changes (migrations, server functions) deploy automatically as soon as they land on `main`.

2. **From your IDE** — push to `main`, then ask the user (or yourself in Lovable chat) to publish. The agent walks through the Pre-Publish checklist and only then calls the publish tool.

Stable URLs:

| Purpose              | URL                                                              |
|----------------------|------------------------------------------------------------------|
| Production (Lovable) | <code>https://signhify.lovable.app</code>                        |
| Production (custom)  | <code>https://signhify.online</code>                             |
| Preview (stable)     | <code>https://project--&lt;project-id&gt;-dev.lovable.app</code> |
| Production (stable)  | <code>https://project--&lt;project-id&gt;.lovable.app</code>     |

Use the stable `project - - ...` URLs for webhooks and cron — they survive project renames.

---

## 7. Self-hosting elsewhere (optional)

Even after wiring local dev, you can still host the build outside Lovable. The repo targets Cloudflare Workers / Vercel Edge / Netlify without modification. See `DEPLOY.md` for variables and adapters.

```
bun run build          # produces .output/
bun scripts/check-deploy.mjs https://your-host.example
```

---

## 8. Troubleshooting

| Symptom                                                  | Likely cause                                        | Fix                                                                                                                              |
|----------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Failed to resolve import "@/..." after git pull          | New file referenced before <code>bun install</code> | Run <code>bun install</code>                                                                                                     |
| Missing Supabase env var(s) in console                   | <code>.env</code> not loaded                        | Restart <code>bun run dev</code> ; verify variable names                                                                         |
| Unauthorized from a <code>createServerFn</code> in SSR   | Protected fn called from a public-route loader      | Call it from the component via <code>useServerFn</code> (see <code>CONTRIBUTING.md</code> )                                      |
| Lovable preview shows old code                           | Push didn't land on <code>main</code>               | Check GitHub Actions; force a re-sync from Lovable's GitHub panel                                                                |
| <code>bun.lockb</code> conflict                          | Two contributors edited deps in parallel            | Delete <code>bun.lockb</code> , re-run <code>bun install</code> , commit                                                         |
| Playwright smoke fails locally                           | Wrong base URL                                      | <code>PLAYWRIGHT_BASE_URL=https://id-preview-<span>&lt;id&gt;</span>.lovable.app</code><br><code>bun run prepublish:check</code> |
| <code>__dirname</code> is not defined in production only | Used a Node-only package in a server fn             | Replace with a Workers-compatible alternative (see server-runtime docs)                                                          |

---

## 9. Golden rules

1. **GitHub is the source of truth.** Both Lovable and your IDE are editors on top of it.
2. **Never hand-edit generated files.** They will be overwritten and merge-conflict at the worst moment.

3. **Secrets live in Lovable Cloud, not in `.env` for production.**

4. **Pre-publish check is non-optional.** It exists because we have shipped broken marketplace diffs before.

5. **Small PRs, fast merges.** The Lovable preview rebuild is fast — use it as a real review surface.

Welcome to the codebase. Ship something.